

Index

Teknisk dokumentation	3
1. Introduktion.....	3
1.1 Projektpresentation	4
1.2 Varför detta system? Vilken problemställning	4
1.3 Systemöversikt: lösningen på problemet	4
1.4 Fil och mappstruktur	4
1.5 Applikationens komponenter	4
1.6 Flöden – flödesschema	5
2. Arbetsmetodik	5
2.1 Iterativt arbete i små steg	5
2.2 ADR (Architecture Design Records)	5
2.3 Journal – documentation under resans gång	6
2.4 Övrig dokumentation	6
2.5 LLM och dess roll i projektet	6
2.6 Hjälp utifrån	6
3. .NET	6
3.1 .NET 8	6
3.2 Blazor Server	7
3.3 ASP.NET Core Identity	7
3.4 Controller-baserad JSON-API	7
3.5 Dependency Injection.....	8
3.6 Patterns	8
3.7 Error handling Helpers	9
3.8 Navigation Helper	9
3.9 Price calculation Helper	9
3.10 Ticket Activation Helper	10
3.11 QR Code Helper	10
3.12 Cosmos JSON Serializer.....	10
3.13 Bibliotek och NuGet-paket	10
4. Patterns.....	11
4.1 Outbox Pattern	11
4.2 Sentinel Key Pattern.....	12

4.3 Repository Pattern	12
4.4 Service Layer Pattern	12
4.5 Extension Methods Pattern	13
4.6 Dependency Injection Pattern	13
4.7 Controller-baserad "REST"-Pattern (JSON-Pattern)	13
4.8 Error Handling Pattern	13
4.9 Feature Flag Pattern.....	14
4.10 Dual-System Coexistence Pattern	14
5. Azure.....	14
5.1 App Service	15
5.2 Cosmos DB	15
5.3 Service Bus	15
5.4 Functions.....	15
5.5 Application Insights	16
5.6 App Configuration	16
5.7 Key Vault	16
5.8 Storage Account	16
5.9 Managed Identity och RBAC	16
5.10 Azure Entry ID.....	16
6. CI/CD	17
6.1 GitHub Actions	17
6.2 Docker	17
6.3 GitHub Container Registry (GHCR)	17
6.4 Bicep (Infrastructure as Code = IaC)	17
6.5 OIDC Authentication	17
6.6 Multi-stage Docker builds	18
6.7 Deployment Strategies	18
6.8 Environment Management	18
7. Säkerhet.....	18
7.1 Testning	18
7.2 Autentisering.....	18
7.3 Säkerhet i Azure.....	19
7.4 GDPR och Session Management	19
7.5 Error Handling och Logging	19
8. Lärdomar och utveckling.....	19
8.1 Lärdomar av projektet.....	19

8.2 Hur produkten kan utvecklas	19
9. Referenser	20
9.1 Microsoft referensmaterial	20

Teknisk dokumentation

1. Introduktion

Fullständigt repo för projektet kan hittas här:

<https://github.com/mymh13/clo24-nikhal78-examwork>

Extensiv dokumentation återfinns i foldern docs. En överblick finns här:

<https://github.com/mymh13/clo24-nikhal78-examwork/blob/main/docs/README.md>

```
docs/
├── README.md           # This file - documentation overview
├── glossary.md         # Glossary of central concepts and acronyms
├── statistics.md       # Project statistics and metrics
├── adr/               # Architecture Decision Records
│   ├── README.md      # ADR index and overview
│   ├── ADR-000-template.md # Template for new Architecture Decision Records
│   ├── ADR-001-cosmosdb.md # Decision: Database choice (Azure Cosmos DB, Serverless)
│   ├── ADR-002-authentication.md # Decision: Authentication via ASP.NET Identity + Entra ID
│   ├── ADR-003-iac.md # Decision: Infrastructure as Code with Bicep
│   ├── ADR-004-frontend.md # Decision: Frontend in .NET 8 Blazor Server
│   ├── ADR-005-azureservices.md # Decision: Core Azure services for operations and monitoring
│   ├── ADR-006-eventdriven.md # Planned decision: Event-driven architecture (Service Bus + Function)
│   ├── ADR-007-ssl-certificate.md # Decision: SSL certificate strategy (manual Let's Encrypt via Docker)
│   ├── ADR-008-docker-deployment.md # Decision: Deployment strategy (Docker containers via GHCR)
│   ├── ADR-009-extension-methods-pattern.md # Decision: Code organization using extension methods pattern
│   ├── ADR-010-gdpr-session-management.md # Decision: GDPR-compliant session management
│   ├── ADR-011-price-calculation-system.md # Decision: Price calculation system with discounts
│   ├── ADR-012-azure-app-configuration.md # Decision: Azure App Configuration for feature flags
│   └── archive/       # Archive for older or replaced ADR documents (currently empty)
├── initial_outtakes/  # Early drafts and plans that defined the project's direction
│   ├── architecture.md # Architectural overview with ASCII diagrams and flow descriptions
│   ├── cosmos-ticketing-considerations.md # Data and domain considerations for booking and ticket model
│   ├── system_overview.md # System overview with project goals, technical plan, and service choices
│   └── Examensarbete-utkast.pdf # Initial thesis draft (PDF)
├── journal/           # Weekly Logs for project progress
│   ├── week_one.md    # Week 1 - startup, planning, and documentation structure
│   ├── week_two.md    # Week 2 - code structure and first runnable version
│   ├── week_three.md  # Week 3 - infrastructure and CI/CD foundation
│   ├── week_four.md   # Week 4 - infrastructure expansion and application integration
│   ├── week_five.md   # Week 5 - feature development and ticket management
│   └── eventdriven_roadmap.md # Detailed roadmap for event-driven architecture refactoring
└── school_related/    # School-related documents (thesis, presentations)
    ├── teknisk_dokumentation_clo24nikhal.docx # Technical documentation (Word)
    └── verktygspresentation_clo24nikhal.docx  # Tool presentation (Word)
```

Det är högst relevant att förstå min dokumentationsstruktur för jag har jobbat iterativt med dokumentering parallellt med att jag bygger projektet. Att återge all den dokumentationen i

denna fil skulle göra att det här dokumentet blir svårläst. Rekommenderar starkt att sätta sig in i min [arbetsmetodik](#) och min dokumentationsstruktur.

1.1 Projektpresentation

- Dual-System Coexistence: Runtime-Switching mellan Synkront och Eventdrivet system
- Biljettbokningssystem för kollektivtrafik
- Toggle-switch för att växla mellan systemarkitektur
- .NET 8 + Azure-molntjänster

1.2 Varför detta system? Vilken problemställning

- Kunden arbetar i två system parallellt (äldre + nyare)
- Behov av ett gränssnitt för att hantera båda systemen samtidigt
- Successiv migration från gammalt till nytt system
- Runtime-växling utan service-restart

1.3 Systemöversikt: lösningen på problemet

- Permanent dual-system coexistence
- Feature flags styr vilket system som används
- Synkront system: direkt API + Cosmos DB (serverless)
- Eventdrivet system: API → Outbox → Service Bus → Azure Functions → Cosmos DB
- Toggle-knappen växlar mellan systemen via Azure App Configuration

1.4 Fil och mappstruktur

Rita upp ASCII-mapp här

- docs/ - Dokumentation (ADR, journal, glossary)
- src/web/ - Blazor Server (UI + API)
- src/functions/ - Azure Functions
- src/shared/ - Delade kontrakt
- src/tests/ - Testprojekt
- infra/ - Bicep-templates
- .github/workflows/ - CI/CD pipelines

1.5 Applikationens komponenter

- Controllers: BookingsController, UsersController, AuthController, HealthController, FeatureFlagController
- Services: BookingService, UserService, OutboxService, FeatureFlagService, TelemetryService, EventPublisher
- Helpers: ErrorHandlerHelper, HttpErrorHelper, PriceCalculationHelper, NavigationHelper, TicketActivationHelper, QrCodeHelper
- Pages: Login, AdminLandingPage, UserLandingPage, Bookings, Users, Health, Demo
- Components: BookingTable, BookingManagement

- Extensions: ServiceCollectionExtensions, WebApplicationExtensions, ConfigurationExtensions

1.6 Flöden – flödesschema

Infoga här ett flödesschema eller liknande

- Synkront flöde:

Användare → Blazor Server → API → Cosmos DB

I det synkrona systemet behandlas bokningen omedelbart. APIet tar emot bokningen, skriver direkt till CosmosDB och svarar sedan tillbaka till användaren. Detta är den enklaste och mest direkta vägen.

Fördelar: Snabb respons, lätt att förstå.

Nackdelar: Mindre robust vid hög belastning, mindre transaktionssäkert.

- Eventdrivet flöde:

Användare → BlazorS → API → Outbox → Service Bus → Azure Functions → Cosmos DB

I det eventdrivna systemet lagras bokningen först som ett meddelande i en Outbox. Sedan skickas det vidare till Service Bus, som i sin tur triggar en Azure Function som utför den slutliga bearbetningen och uppdaterar Cosmos DB.

- Toggle-switch: Feature flag i App Configuration styr vilket flöde som används
- Outbox Pattern: Säkerställer transaktionssäkerhet vid dual writes

2. Arbetsmetodik

Tycker det är högst relevant att ta med!

2.1 Iterativt arbete i små steg

- Arbete i faser med delmoment
- Varje fas bröts ner i små, hanterbara delar
- Simulerar agilt arbete utan formella sprints
- Kontinuerlig refaktorering och förbättring
- Testning integrerad i varje fas

2.2 ADR (Architecture Design Records)

- Vad är ADR
- Dokumentation av tekniska beslut
- Motivering till varje val
- Konsekvenser och alternativ
- 20 ADR-dokument skapade

- Viktiga ADRer: ADR-001 (Cosmos DB), ADR-004 (Blazor Server), ADR-005 (Azure Services), ADR-013 (Outbox Pattern), ADR-014 (Sentinel Key)

2.3 Journal – documentation under resans gång

- Veckovis loggning av framsteg
- Dokumentation av aktiviteter, utmaningar och lärdomar
- Timeline för projektet
- Veckor 1-6 dokumenterade
- Event-driven roadmap för detaljerad implementation

2.4 Övrig dokumentation

- Glossary - Förklarar begrepp och termer
- Systemöversikt - Övergripande bild av systemet
- Architecture.md - Arkitekturdiagram och flöden
- Bug backlog - Kända buggar och lösningar
- Future improvements - Framtida förbättringar

2.5 LLM och dess roll i projektet

- Dokumentation under kodning
- Arkitektur och designstöd
- Generera kod, agent

2.6 Hjälp utifrån

- Handledare LIA – Henrik: Sentinel Key
- Blog – Tore Nestenius: SessionStore cookie-hantering - <https://nstenius.se/net/improving-asp-net-core-security-by-putting-your-cookies-on-a-diet/>
- Tips från Nätverket – Mats Alritzson m.fl

3. .NET

Detta projektet använder .NET 8, Microsofts plattform för att bygga fullskaliga applikationslösningar. Plattformen erbjuder ett enhetligt ekosystem där backend, frontend, testning, konfiguration och säkerhet kan byggas med samma språk och samma infrastruktur.

Detta ger minskad komplexitet, förbättrad underhållbarhet (i synnerhet som kunden idag jobbar med .NET-verktygen), och i just det här fallet är det även lättare att integrera ny kod in i gamla strukturer.

3.1 .NET 8

Varför .NET 8 nu när .NET 10 finns ute?

Kunden kör redan .NET 8 i sin miljö idag. Det är ändå ett relativt modernt system med LTS-support. Bra stöd i Azure-tjänsterna och ett gemensamt språk (C#) för både frontend och backend. Det är ett tekniskt modernt, hållbart och ekonomiskt fördelaktigt val för kunden.

3.2 Blazor Server

Blazor Server är det UI-ramverk som används för att bygga gränssnittet i projektet. Det är serverdrivet och bygger på SignalR för realtidskommunikation, vilket gör att gränssnittet kan uppdateras direkt utan sidladdningar.

Blazor använder Razor Components för att strukturera UI:t i tydliga och återanvändbara moduler, och kräver inget JavaScript för kärnfunktionaliteten. Det ger en snabb och lättunderhållen lösning där både frontend och backend delar samma språk (C#).

Varför Blazor Server i detta projekt:

Det används redan i kundens befintliga .NET-miljö, är lätt att integrera med API:erna, kräver minimalt med externa beroenden och erbjuder hög utvecklingshastighet vid iterativt arbete. Men rent krasst, det var en av de techstackarna som fanns med i kravspecen. Jag kan personligen tycka att HTMX hade passat för den här typen av applikation.

3.3 ASP.NET Core Identity

Projektet använder ASP.NET Core Identity för användarhantering och autentisering. Identity tillhandahåller ett stabilt ramverk för inloggning, lösenordshantering och rollbaserad åtkomstkontroll. Systemet använder cookie-baserad autentisering för vanliga användare, kombinerat med BCrypt för säker lösenordshashning.

Rollerna i projektet – Admin, Inspector och User – styr vilka funktioner varje användare får tillgång till, och säkerställer en tydlig behörighetsmodell.

Varför Identity i detta projekt:

Det är en beprövad säkerhetslösning, fullt integrerad i .NET-ekosystemet och kompatibel med både kundens miljö och Azure-tjänsterna. Det ger en grundläggande säkerhet utan behov av externa system eller egen implementation. Huvudskälet var även att det var otroligt lätt att integrera, när EntraID satt för Admin-rollen så behövdes en smidig men ändå tydlig säkerhetslösning för Inspector och Users login.

3.4 Controller-baserad JSON-API

Projektet exponerar sitt backend genom ett controller-baserat JSON-API i ASP.NET Core. Designen följer principerna för REST-liknande API:er: resurser hanteras via standardiserade HTTP-metoder (GET, POST, PUT, DELETE), och data utbyts i JSON-format.

API-et består av flera separata controllers, bland annat BookingsController, UsersController, AuthController, HealthController och FeatureFlagController, vilket ger en tydlig ansvarsfördelning och enkel testbarhet.

Felhantering är centraliserad genom *ErrorHandlerHelper*, som säkerställer konsekventa HTTP-svar och strukturerad felinformation.

Varför denna design i projektet:

Den är enkel att implementera, lätt att förstå, följer etablerad .NET-praxis och passar projektets behov av tydliga endpoints för UI, adminfunktionalitet och eventdrivna processer. Jag valde även denna design delvis för att vissa av Controllerna är återanvändbara i andra roller (Inspector har vissa rättigheter som Admin har, exempelvis).

3.4.1 Om REST vs. JSON-API (kort sidonot):

Systemet använder JSON-baserad kommunikation i vad som populärt men felaktigt kallas "REST"-API. För att bättre förstå skillnaden mellan REST, "REST-lik" API och varför vi egentligen borde kalla det vi kallar "REST-API" för JSON-API, se [REST explained for beginners](#).

3.5 Dependency Injection

Projektet använder .NET:s inbyggda Dependency Injection (DI) för att strukturera och organisera applikationens beroenden. DI-containern hanterar instansiering av tjänster och gör det möjligt att registrera dem med olika livscykler (Scoped, Singleton, Transient) beroende på behov.

All funktionalitet exponeras via interface-baserad design, vilket ger löskoppling, förbättrar testbarheten och undviker täta eller onödiga beroenden mellan komponenter. Registrering av tjänster sker genom tydliga extension methods, vilket är min favorit-goto-pattern om Program.cs börjar växa sig för långt.. det är ett högst personligt val för att få en tydligare överblick av körfilen!

Varför DI i detta projekt:

Det är standard i modern .NET-utveckling, underlättar testning, möjliggör enkel mockning och bidrar till en ren arkitektur där ansvar är tydligt separerat.

3.6 Patterns

Applikationen använder flera etablerade arkitekturmönster för att skapa en tydlig, testbar och modular struktur. Nedan listas de patterns som används i projektet, med hänvisning till respektive avsnitt där de beskrivs mer ingående:

- [Repository Pattern](#) – se avsnitt 4.3
- [Service Layer Pattern](#) – se avsnitt 4.4
- [Extension Methods Pattern](#) – se avsnitt 4.5
- [Dependency Injection Pattern](#) – se avsnitt 4.6
- [Controller-baserad REST/JSON Pattern](#) – se avsnitt 4.7
- [Error Handling Pattern](#) – se avsnitt 4.8
- [Feature Flag Pattern](#) – se avsnitt 4.9
- [Dual-System Coexistence Pattern](#) – se avsnitt 4.10
- [Managed Identity Pattern](#) – se avsnitt 4.11
- [Telemetry Pattern](#) – se avsnitt 4.12

Syfte med patterns:

De skapar en enhetlig arkitektur, minskar komplexitet och underlättar både testning och vidareutveckling. Detaljer och tekniska motiv presenteras i respektive avsnitt. Patterns ger

också en tydlighet för utvecklare som vill sätta sig in i kodbasen, kan de patternen så kommer de snabbare förstå kodbasen.

3.7 Error handling Helpers

Projektet använder två hjälpbibliotek för att hantera fel på ett strukturerat och konsekvent sätt:

- `ErrorHandlerHelper` – centraliserad felhantering för API-controllern, ansvarar för att generera standardiserade HTTP-svar och loggposter.
- `HttpErrorHandler` – motsvarande hantering för Razor Pages, med fokus på användarvänliga felmeddelanden och korrekt återkoppling i UI:t.

Dessa helpers säkerställer att fel presenteras på ett enhetligt sätt, att ingen känslig information läcker ut, och att all viktig information loggas strukturerat för analys i Application Insights.

Varför detta i projektet:

Det förenklar felsökning, förbättrar användarupplevelsen, bidrar till en robust driftmiljö. Jag implementerade de här hjälpsystemen när jag insåg att jag började upprepa error handling i olika kodfiler, att centralisera hanteringen var ett naturligt val då.

3.8 Navigation Helper

`NavigationHelper` ansvarar för den rollbaserade navigeringen i applikationen. Den centrala funktionen `GetLandingPageUrl()` returnerar rätt landningssida baserat på användarens roll, exempelvis Admin, Inspector eller User.

Hjälparen används primärt i *Login.razor* men återkommer även i andra sidor där applikationen behöver styra användaren till korrekt vy efter inloggning eller vid rollbyte.

Varför detta i projektet:

Det ger en tydlig, centraliserad logik som säkerställer konsekvent navigationsbeteende och minskar duplicering i UI-komponenterna. Den kom till när jag ville skapa ett system för att återkomma till din roll-baserade landing page, men inte ville duplicera kod för varje roll.

3.9 Price calculation Helper

`PriceCalculationHelper` ansvarar för all logik kopplad till biljettprissättning. Den hanterar zonbaserad prissättning, åldersbaserade rabatter (barn, vuxen, pensionär), studentrabatt samt ett konfigurerbart baspris per zon.

Genom att samla prissättningsreglerna i en central helper hålls både UI och tjänstelager fria från duplicerad logik, vilket gör förändringar och tester betydligt enklare.

Varför detta i projektet:

Det säkerställer konsekventa prisberäkningar över hela systemet och ger en strukturerad plats för framtida ändringar i prissättning och affärsregler. Men skälet till att denna blev till var helt enkelt för att inte ha ett hårdkodat pris i koden. Med detta verktyg blir det lättare att manipulera kostnaden och bli mer dynamisk, och/eller prisjustera.

3.10 Ticket Activation Helper

TicketActivationHelper hanterar logiken kring aktivering av biljetter. Den validerar om en biljett kan aktiveras, beräknar giltighetsperioden (90 minuter) och utför statuskontroller mot biljettens livscykel: Created, Activated, Valid och Expired.

Kärnfunktionerna *CanActivate()* och *ValidateActivation()* används av både UI och tjänstelager för att säkerställa att aktivering sker korrekt och enligt regelverket.

Varför detta i projektet:

Det centraliserar all aktiveringslogik, minskar duplicering och gör systemet lättare att testa samt enklare att vidareutveckla vid förändrade affärsregler.

3.11 QR Code Helper

QrCodeHelper använder QRCode-biblioteket för att generera QR-koder kopplade till biljetter. QR-koderna skapas som PNG-bilder, kodas till Base64-strängar och lagras i Cosmos DB tillsammans med JSON-strukturerad data (bland annat Booking ID, Customer ID, tidsstämplar och status).

Funktionen GetQrCodeDataUrl() används för att exponera QR-koden direkt i UI:t via en data:-URL, utan behov av separat filhantering.

Varför detta i projektet:

Det ger en enkel lösning för att koppla varje biljett till en unik, maskinläsbar identifierare som är lätt att visa i gränssnittet och möjlig att validera om biljettinspektören implementeras.

3.12 Cosmos JSON Serializer

Projektet använder en anpassad JSON-serializer för Cosmos DB för att hantera de detaljer och begränsningar som följer med databasen, exempelvis property-namn, typer och systemfält. Lösningen bygger på System.Text.Json, vilket ger hög prestanda och god kontroll över serialiseringsprocessen.

Denna serializer säkerställer att dokumentformatet är konsekvent, kompatibelt med Cosmos DB:s krav och tillförlitligt vid både läsning och skrivning.

Varför detta i projektet:

Det eliminerar serialiseringsproblem och skapar en stabil datamodell som är enkel att felsöka och vidareutveckla. Jag fick problem med serialiseringen av enums och då föddes den här lösningen. Jag har noterat i dokumentationen att enum har sina styrkor, men även medför risker just när det gäller serialisering. Här fick jag värdefulla tips av min handledare på LIAn.

3.13 Bibliotek och NuGet-paket

Projektet använder ett antal centrala NuGet-paket och bibliotek som stödjer funktionalitet, konfiguration och integrationer:

- QRCode – generering av QR-koder för biljetter
- System.Text.Json – snabb och flexibel JSON-serialisering

- Microsoft.Extensions.* – kärnbibliotek för konfiguration, dependency injection och logging
- Azure.* – officiella SDK-paket för Cosmos DB, Service Bus, App Configuration och Key Vault

Tillsammans ger dessa paket en modern, väl underhållen/supportad och fullt integrerad teknisk bas för applikationen.

4. Patterns

Applikationen använder flera etablerade arkitekturmönster för att skapa en tydlig struktur. Varje pattern adresserar ett specifikt problem. Vinsten med att använda dem blir både i form av att koden blir lättare att läsa, men hjälper även till om kodbasen skalar upp i storlek.

Nedan följer de patterns som använts i projektet, tillsammans med en kort beskrivning av vad de löser och varför de valts. Mer djupgående beskrivningar finns i respektive underrubrik.

4.1 Outbox Pattern

Outbox Pattern används för att garantera att databasoperationer och eventpublicering sker på ett säkert och konsekvent sätt, särskilt när samma händelse behöver skrivas både till Cosmos DB och publiceras till Service Bus. Utan detta uppstår det klassiska *dual write*-problemet, där en av operationerna kan lyckas medan den andra misslyckas.

I projektet implementeras Outbox Pattern genom:

- En outbox-container i Cosmos DB där events lagras med status (Pending, Processed, Failed).
- Alla bokningar skapar ett event atomiskt tillsammans med databasskrivningen.
- En bakgrundstjänst (OutboxProcessorService) pollar kontinuerligt outboxen och publicerar events till Service Bus.
- Retry-logik med exponential backoff säkerställer att tillfälliga fel hanteras robust.
- Audit trail bibehålls genom att alla events sparas oavsett utfall.

I det synkrona systemet skapas eventet men publiceras inte.

I det eventdrivna systemet publiceras eventet till Service Bus, som sedan triggar Azure Functions för vidare bearbetning.

Varför Outbox Pattern i detta projekt:

Det säkerställer konsekvent databehandling i en arkitektur där både synkrona och eventdrivna flöden måste fungera parallellt, utan risk för inkonsistens mellan databasen och kö-systemet.

4.2 Sentinel Key Pattern

Sentinel Key Pattern är centralt i projektet eftersom systemet ska kunna växla mellan synkront och eventdrivet läge i realtid, utan driftstopp.

I detta projekt implementeras mönstret genom:

- En särskild trigger-nyckel, Settings.Sentinel, i Azure App Configuration.
- När denna nyckel ändras uppdateras alla konfigurationsvärden automatiskt.
- Ett refresh-intervall på 30 sekunder, kompletterat med middleware som anropar TryRefreshAsync() vid varje HTTP-request.
- Full zero-downtime switching, vilket innebär att applikationen omedelbart kan ta nya inställningar i bruk.

Varför Sentinel Key Pattern i detta projekt:

Det möjliggör den centrala funktionaliteten i lösningen: runtime-växling mellan två systemarkitekturer utan avbrott. Mönstret är dessutom avgörande för demo-scenariot där systemet ska uppdateras live inför publik.

Vill notera att jag hade initialt tänkt att köra Feature Flags, men kände inte till Sentinel Key så då hade jag förväntat mig att få starta om servicen mellan varje toggle. Jag pratade med min handledare på LIAn, [Henrik Jönsson](#), och han tipsade om Sentinel Key. Detta är första gången jag implementerar en sådan lösning.

4.3 Repository Pattern

Repository Pattern används för att abstrahera dataåtkomst och skapa en tydlig separation mellan applikationslogik och datalager. I projektet definieras gränssnitt som IBookingService, IUserService och IOutboxService, vilket gör det möjligt att byta datakälla eller implementering utan att resten av systemet påverkas.

Mönstret förenklar även testning, eftersom mock- och in-memory-implementationer kan användas utan beroenden till Cosmos DB eller andra externa system.

Varför detta pattern i projektet:

Det ger renare arkitektur, bättre testbarhet och minskar kopplingen mellan affärslogik och dataåtkomst.

4.4 Service Layer Pattern

Service Layer Pattern strukturerar applikationens affärslogik i separata serviceklasser, vilket gör att controllers kan fokusera på HTTP-hantering och input/output-modeller. Den komplexa logiken ligger i services såsom BookingService, UserService och OutboxService, vilket skapar en tydlig ansvarsfördelning och förbättrar läsbarheten i hela koden.

Notera symbiosen det skapar med Repository Pattern ovan. Isolerad affärslogik. Säkrar data.

Varför detta pattern i projektet:

Det möjliggör en modulär och utbyggbar arkitektur, förenklar felsökning och gör det enklare att testa affärslogik isolerat från UI och controllers.

4.5 Extension Methods Pattern

Extension Methods Pattern används för att organisera applikationens startup- och konfigurationslogik i separata, återanvändbara metoder. I projektet delas detta upp i exempelvis `ServiceCollectionExtensions`, `WebApplicationExtensions` och `ConfigurationExtensions`, vilket skapar en mer strukturerad och läsbar *Program.cs*.

Genom att samla registreringar och konfigurationssteg i tydligt namngivna extension methods får projektet en modulariserad uppstart med hög återanvändbarhet.

Det här är ett favorit-pattern hos mig, det gör att *Program.cs* blir lättare att läsa, vi ser tydligt körordning (vilket kan vara relevant ibland), vi ser direkt vilka services som är kopplade till vilka funktioner, etc.

Varför detta pattern i projektet:

Det ger en renare kodbas, enklare navigation och minskar komplexitet i applikationens startsekvens. Det löser inget problem för kunden, men det gör utvecklarens jobb lättare.

4.6 Dependency Injection Pattern

Dependency Injection Pattern är centralt i .NET och används genomgående i projektet. Genom interface-baserad design och constructor injection kan tjänster med olika livscykler (Scoped, Singleton, Transient) registreras och tilldelas på ett kontrollerat sätt.

Patternet kompletterar [Repository](#)- och [Service Layer](#)-mönstren (se 4.3 och 4.4) och gör det möjligt att ersätta implementationer i testmiljöer samt minska kopplingen mellan komponenter.

Varför detta pattern i projektet:

Det ger en flexibel arkitektur som är lätt att testa, underhålla och vidareutveckla, samtidigt som det följer etablerade .NET-principer.

4.7 Controller-baserad "REST"-Pattern (JSON-Pattern)

Detta pattern används för att strukturera API:et enligt välkända [så kallade REST](#)-principer, där resurser exponeras genom standardiserade HTTP-metoder (GET, POST, PUT, DELETE) och data utbyts i JSON-format.

Applikationen följer en konsekvent URL-struktur och använder HTTP-statuskoder för att signalera utfall vid varje operation, vilket förenklar både implementation och felsökning.

Varför detta pattern i projektet:

Det ger en tydlig, förutsägbar API-design som passar .NET:s controller-modell och fungerar väl tillsammans med både Blazor Server och externa integratörer.

4.8 Error Handling Pattern

Error Handling Pattern centraliserar felhantering i applikationen och säkerställer att API och UI svarar på ett enhetligt sätt vid fel.

Projektet använder ErrorHandlerHelper för controllers och HttpErrorHelper för Razor Pages, vilket ger konsekventa felmeddelanden, strukturerad logging och en förbättrad användarupplevelse.

Varför detta pattern i projektet:

Det minskar risken för inkonsekvent felhantering, underlättar felsökning och säkerställer att inga känsliga detaljer läcker ut i felresponsen. Gillade även att samla all felkodshantering under ett tak, det blir lättare att hitta och hjälper till mot möjlig duplicering av kod.

4.9 Feature Flag Pattern

Feature Flag Pattern används för att aktivera och styra funktionalitet i runtime utan omstart av applikationen. I projektet hanteras detta via Azure App Configuration, där flaggan BookingEvents_Enabled avgör om systemet ska köras i synkront eller eventdrivet läge.

Flaggan uppdateras dynamiskt genom [Sentinel Key Pattern](#) (se 4.2), vilket gör att växlingen kan ske direkt. Admin-gränssnittet innehåller en toggle-knapp som ändrar flaggans värde i realtid.

Varför detta pattern i projektet:

Det möjliggör flexibel konfiguration, live-demonstrationer och driftsättning av nya funktioner utan avbrott.

4.10 Dual-System Coexistence Pattern

Dual-System Coexistence Pattern beskriver arkitekturen där två olika systemflöden, i detta fall ett synkront och ett eventdrivet, kan köras parallellt och växlas mellan i realtid.

Feature-flaggan styr aktivt system, men båda flödena existerar och fungerar sida vid sida. Outbox är alltid aktiv och loggar event oavsett vilket system som används. Den synkrona modellen fortsätter fungera oförändrad, vilket gör att inga brytande ändringar införs vid växling.

Varför detta pattern i projektet:

Det möjliggör en permanent arkitektur där både äldre och nyare systemlogik kan samexistera, vilket är nödvändigt för kundens successiva migreringsbehov.

5. Azure

Azure utgör molnplattformen för hela applikationen och ansvarar för hosting, databashantering, meddelandeflöden, konfigurationsstyrning och säker åtkomstkontroll. Projektet använder en uppsättning integrerade Azure-tjänster som tillsammans möjliggör den duala systemarkitekturen (synkront + eventdrivet), säker drift, hög flexibilitet och kostnadseffektiv utveckling.

Varje tjänst fyller en specifik funktion och är antingen vald för att den redan finns i kundens existerande system, eller för att fyller en specifik funktion i det eventdrivna flödet.

5.1 App Service

Azure App Service används som hostingplattform för Blazor Server-applikationen och dess API. Projektet körs i en Basic B1-plan, baserad på Linux med .NET 8-support.

Applikationen distribueras som [Docker](#)-containers direkt [från GitHub Container Registry \(GHCR\)](#), vilket möjliggör förutsägbara deployments och versionshantering.

App Service är dessutom kopplad till Azure via Managed Identity, vilket ger säker åtkomst till Cosmos DB, App Configuration och Service Bus utan att några secrets eller connection strings behöver hanteras i kod.

Varför App Service i projektet:

Det är en stabil, kostnadseffektiv och fullt managerad hostinglösning som passar utmärkt för våra behov i detta projekt.

5.2 Cosmos DB

Cosmos DB används som applikationens primära datalager. Det är en serverless NoSQL-databas, vilket innebär att kostnaden skalar baserat på faktisk användning, inte reserverad kapacitet. Datamodellen består av tre containers: bookings, users och outbox, där outboxen stödjer eventdrivna flöden och auditloggning.

Partition keys används för att möjliggöra effektiv och förutsägbar query-prestanda, och databasen konfigureras med transactional consistency, vilket säkerställer att bokningar och relaterade event skrivs på ett tillförlitligt sätt.

Varför Cosmos DB i projektet:

Den erbjuder låg latency, flexibel datamodellering, serverless billing och direkt integration med Managed Identity, vilket gör den idealisk för ett skalbart biljettsystem. Varför serverless? En vanlig databas med reserverad kapacitet har en fast kostnad. Serverless skalar efter behov, men framför allt, kostar bara vid användning.

5.3 Service Bus

- Meddelandekö för event-driven architecture
- booking-events queue
- Dead letter queue support
- Basic tier (dev)
- Managed Identity access

5.4 Functions

- Serverless event processing
- Processar Service Bus-meddelanden
- Basic B1 plan (dev)
- .NET 8 isolated worker
- Managed Identity för Cosmos DB och Service Bus

5.5 Application Insights

- Logging, monitoring, performance measurement
- KQL-queries för analys
- Custom events för dual-system architecture
- Telemetry abstraction layer (ItelemetryService interface)
- Custom events: BookingCreated, OutboxEventCreated, OutboxEventProcessed, ServiceBusEventPublished, FeatureFlagToggled, ModeSwitch
- Custom dimensions: SystemType, ToMode, ArchitectureMode för att skilja synkront/eventdrivet
- Workbooks för visualisering (Current Mode Indicator, Latest Booking Flow Timeline, Events by Type)
- Pay-as-you-go pricing
- Används av både App Service och Function App

5.6 App Configuration

- Centraliserad konfigurationshantering
- Feature flags (BookingEvents_Enabled)
- Hot-reload via Sentinel Key Pattern
- Free tier (dev)
- Managed Identity access

5.7 Key Vault

- Säker lagring av secrets och connection strings
- Standard tier
- RBAC-baserad åtkomst via Managed Identity
- Inga secrets i kod eller konfigurationsfiler

5.8 Storage Account

- Krävs för Azure Functions runtime
- State management för Functions
- Standard LRS
- Används indirekt av Function App

5.9 Managed Identity och RBAC

- Alla tjänster använder Managed Identity
- RBAC-baserad autentisering
- App Service: "App Configuration Data Reader", "Azure Service Bus Data Owner"
- Function App: "Azure Service Bus Data Receiver", "DocumentDB Account Contributor"
- Inga connection strings i kod

5.10 Azure Entry ID

- Autentisering för Admin/Inspector-roller

- Microsoft-konto integration
- OAuth 2.0 / OpenID Connect

5.11 Kostnader

- Driftskostnader november
- Driftskostnader december
- Bryter ner kostnaderna sektion för sektion och en överblick

6. CI/CD

Övergripande bild av CI/CD

6.1 GitHub Actions

- CI/CD-plattform
- YAML-baserade pipelines
- Separated CI/CD (build och deploy i separata workflows)
- OIDC-autentisering för Azure
- Environment protection (dev/prod)

6.2 Docker

- Containerisering av applikationen
- Multi-stage builds för optimering
- Löste Oryx auto-detection problem
- Dockerfile för både web och functions

6.3 GitHub Container Registry (GHCR)

- Container-lagring
- Public images för dev
- Integrerat med GitHub Actions
- Pull images i Azure App Service

6.4 Bicep (Infrastructure as Code = IaC)

- Azure-infrastruktur som kod
- Modulär struktur - återanvändbara moduler
- Resource Groups - separata för dev/prod
- Managed Identity konfiguration
- RBAC-roller definierade i Bicep

6.5 OIDC Authentication

- Säker Azure-autentisering utan credentials

- GitHub Actions använder OIDC för Azure login
- Federated identity credentials
- Inga secrets i GitHub Actions

6.6 Multi-stage Docker builds

- Optimering av container-storlek
- Build stage och runtime stage
- Separerade dependencies
- Snabbare deployments

6.7 Deployment Strategies

Maybe mention: Scaled Trunk?

- Web App: Docker-containers via GHCR
- Function App: Zip-deploy
- Idempotent deployments
- Rollback-möjligheter

6.8 Environment Management

- Separata environments (dev/prod)
- Environment protection rules
- GitHub Environments för deployment approval
- Separata Resource Groups per environment

7. Säkerhet

Övergripande bild av säkerhet

7.1 Testning

- xUnit - Testramverk
- NSubstitute - Mocking
- Unit tests - Affärslogik (price calculations)
- Integration tests - Fullstack med WebApplicationFactory
- In-memory mocks - InMemoryUserService, InMemoryBookingService, InMemoryOutboxService
- InMemoryStorage - Singleton för delad data i tester

7.2 Autentisering

- Azure Entra ID för Admin/Inspector
- Cookie Authentication för vanliga användare

- ASP.NET Core Identity-principer
- BCrypt för lösenordshashning
- Rollbaserad åtkomst (RBAC)

7.3 Säkerhet i Azure

- Managed Identity överallt
- RBAC-baserad åtkomst
- Inga connection strings i kod
- Key Vault för secrets
- Network security (kan utökas med private endpoints)

7.4 GDPR och Session Management

- Server-side session storage
- Cookie-baserad autentisering
- Säker hantering av användardata
- ADR-010 dokumenterar GDPR-kompatibel session management

7.5 Error Handling och Logging

- Centraliserad felhantering
- Inga känsliga detaljer exponeras
- Strukturerad logging med Application Insights
- User-friendly error messages

8. Lärdomar och utveckling

Introduktion

8.1 Lärdomar av projektet

- Docker löste Oryx-problem - Bypassade auto-detection
- Sentinel Key möjliggjorde live-demos - Runtime-switching kritiskt
- Outbox Pattern säkrar data - Dual write-problem kräver transaktionssäkerhet
- In-memory mocks för tester - Snabbare, isolerade tester
- Extension Methods förbättrar läsbarhet - Startup-kod tydligare
- Centraliserad error handling - Konsistent UX och logging
- Component extraction - Eliminerade kodduplicering
- Configuration as code - Runtime-anpassningar möjliga
- MVP-filosofi - Fokus på kärnfunktionalitet, undvik over-engineering
- Right tool for the job - Azure Portal för Application Insights visualisering

8.2 Hur produkten kan utvecklas

- API Management (APIM) - Gateway för publika endpoints

- Cosmos DB Change Feed - Push-baserad event-processing
- Event-driven ticket expiration - Automatisk expirering
- Shopping Cart - Lägg till flera biljetter innan betalning
- User Registration - Användarregistrering i UI
- QR Code Scanning - Fysisk scanning för Inspectors
- Ticket Search - Sök och filtrera i admin-bokningar
- A/B testing - Feature flags för gradual rollouts
- Multi-environment - Staging och production
- Rate Limiting - Skydd för API-endpoints
- Swagger/OpenAPI - API-dokumentation
- Domain Separation - Separera domäner för bättre arkitektur

9. Referenser

Introduktion?

9.1 Microsoft referensmaterial

Länkar till Microsoft Learn:

- Microsoft Docs - Azure Services
- Microsoft Docs - .NET 8
- Microsoft Docs - Blazor Server
- Microsoft Docs - Application Insights
- Microsoft Docs - Cosmos DB
- Microsoft Docs - Service Bus
- Microsoft Docs - Azure Functions
- Microsoft Docs - Bicep

The end?